

HW 3 Core: Balcony Trajectory

Week 3 — fragment submission

Core homework: a throw from a balcony

We extend the lecture's sampled projectile to user-supplied initial height, speed, angle, and positive timestep. We keep $+y$ upward and advance the old state with explicit Euler. We store aligned histories of time, position, and both velocity components, including the initial state and the first sample with $y \leq 0$.

We proceed in four stages. First we sketch the expected trajectory, state units and signs, and calculate an analytic landing time for comparison. Next we construct the vectors and write the loop, checking the first update by hand. Then we verify that every vector has the same length, time increases by the specified step, horizontal velocity remains constant, and vertical velocity decreases by $g\Delta t$. Finally we repeat with half the timestep and tabulate the sampled landing times and ranges beside the analytic values.

The report must distinguish a ground-crossing sample from an interpolated or exact impact. We do not clip the final negative height or silently replace the numerical landing with the analytic answer. We follow the assignment's initial-ground convention, including the upward-launch case, rather than assuming that `while y[end] > 0` handles every initial condition. The homework sheet spells out the numerical fragment, the public checks, and the separately assessed trajectory and refinement artifacts. Function definitions remain Week 5 material.

Physics box: sampled projectile motion

A balcony throw is a point projectile under approximately uniform gravity with no air resistance. We take $+x$ horizontal and $+y$ upward, so $a_x = 0$ and $a_y = -g$ with $g > 0$ in m/s^2 ; the state is (t, x, y, v_x, v_y) with time in seconds, positions in metres, and velocities in m/s . Explicit Euler advances the state by $x_{n+1} = x_n + v_{x,n}\Delta t$, $y_{n+1} = y_n + v_{y,n}\Delta t$, then $v_{x,n+1} = v_{x,n}$ and $v_{y,n+1} = v_{y,n} - g\Delta t$, so position uses the old velocity. The stored final point is the first sampled height with $y \leq 0$, not the exact ground crossing. The lecture case $v_0 = 10 \text{ m/s}$, $\theta = 45^\circ$, $y_0 = 1 \text{ m}$, $g = 9.81 \text{ m/s}^2$ has analytic landing time about 1.57135 s and range about 11.111 m; halving Δt should move the sampled values toward these. Refining the step checks the discretization only and cannot add the omitted drag, bounce, or wall impact.

Task

We simulate a projectile launched from a balcony and store its sampled history. Use $+x$ horizontally and $+y$ upward, $a_x = 0$, $a_y = -g$, and forward Euler updates: update position from the old velocity, then update velocity. Start with the initial state and append samples until including the first sample with `y[end] <= 0`. Store aligned vectors `t`, `x`, `y`, `vx`, and `vy`; assign `landing_time = t[end]`. The result is a sampled landing, not an exact impact; the no-drag, uniform-gravity model omits collision and air resistance.

Validate that the supplied names are finite non-Boolean reals, then require `height_m >= 0`, `speed_m_s >= 0`, `dt_s > 0`, and `gravity_m_s2 > 0`. (Julia treats `Bool` as a `Real`, so reject it explicitly.) Invalid input raises `ArgumentError`. For a zero-height launch, the initial state is also the required first `y <= 0` sample.

Submission

Submit this fragment only; functions begin in Week 5.

```
# validate inputs; initialize and fill the five vectors
# assign landing_time
```

Checking

Before we call the task done, it helps to check what a correct result has to satisfy. The five vectors should all have the same length, time should begin at `t[1] == 0.0` and advance by `dt_s`, and the last stored height should be the first nonpositive one. Run the fragment once with `dt_s` and once with `dt_s / 2`, then compare the two sampled landing times and ranges; that is how we see how much the answer moves when the step shrinks. Refining the timestep tests that sensitivity only—it cannot recover the collision or air-resistance physics our model leaves out.

Optional advanced wrapper and tests

The supplied wrapper is `balcony_trajectory(height_m, speed_m_s, angle_deg, dt_s; gravity_m_s2=9.81)`, and it returns `(t, x, y, vx, vy, landing_time)`. If we want to see how the platform checks our work, the optional reading brings in using `Test: @test` states a claim that must hold, and `@test_throws ArgumentError` states that a documented invalid input must be rejected. Seeing those claims, and running them ourselves, is what makes the check reproducible rather than a matter of trust. The staff tests use this published contract only, never adding requirements of their own. The complete public test body is:

```
run_question_tests("week03 core public") do
  t, x, y, vx, vy, landing = balcony_trajectory(2.0, 10.0, 45.0, 0.1)
  @test length(t) == length(x) == length(y) == length(vx) == length(vy)
)
@test t[1] == 0.0 && y[1] == 2.0 && landing == t[end]
@test y[end] <= 0 && y[end - 1] > 0
@test all(isapprox.(diff(t), 0.1))
@test balcony_trajectory(0.0, 5.0, 30.0, 0.1)[1:3] == ([0.0], [0.0], [0.0])
@test_throws ArgumentError balcony_trajectory(-1.0, 1.0, 0.0, 0.1)
@test_throws ArgumentError balcony_trajectory(1.0, 1.0, 0.0, 0.0)
@test_throws ArgumentError balcony_trajectory(true, 1.0, 0.0, 0.1)
end
```